Creating and Managing Database Objects

SNOWFLAKE ARCHITECTURE

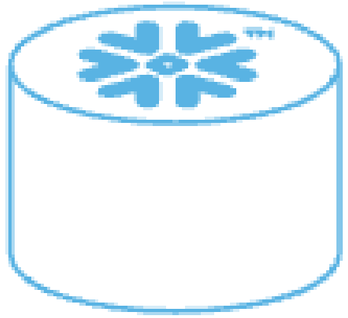
Emily Melhuish
Technical Curriculum Developer,
Snowflake

datacamp

Creating a Database and Schema

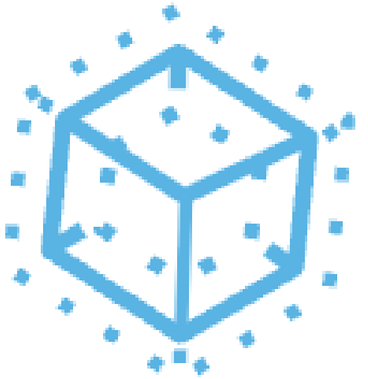
Creating a Database Query

```
CREATE DATABASE snowy_peak_db;
```

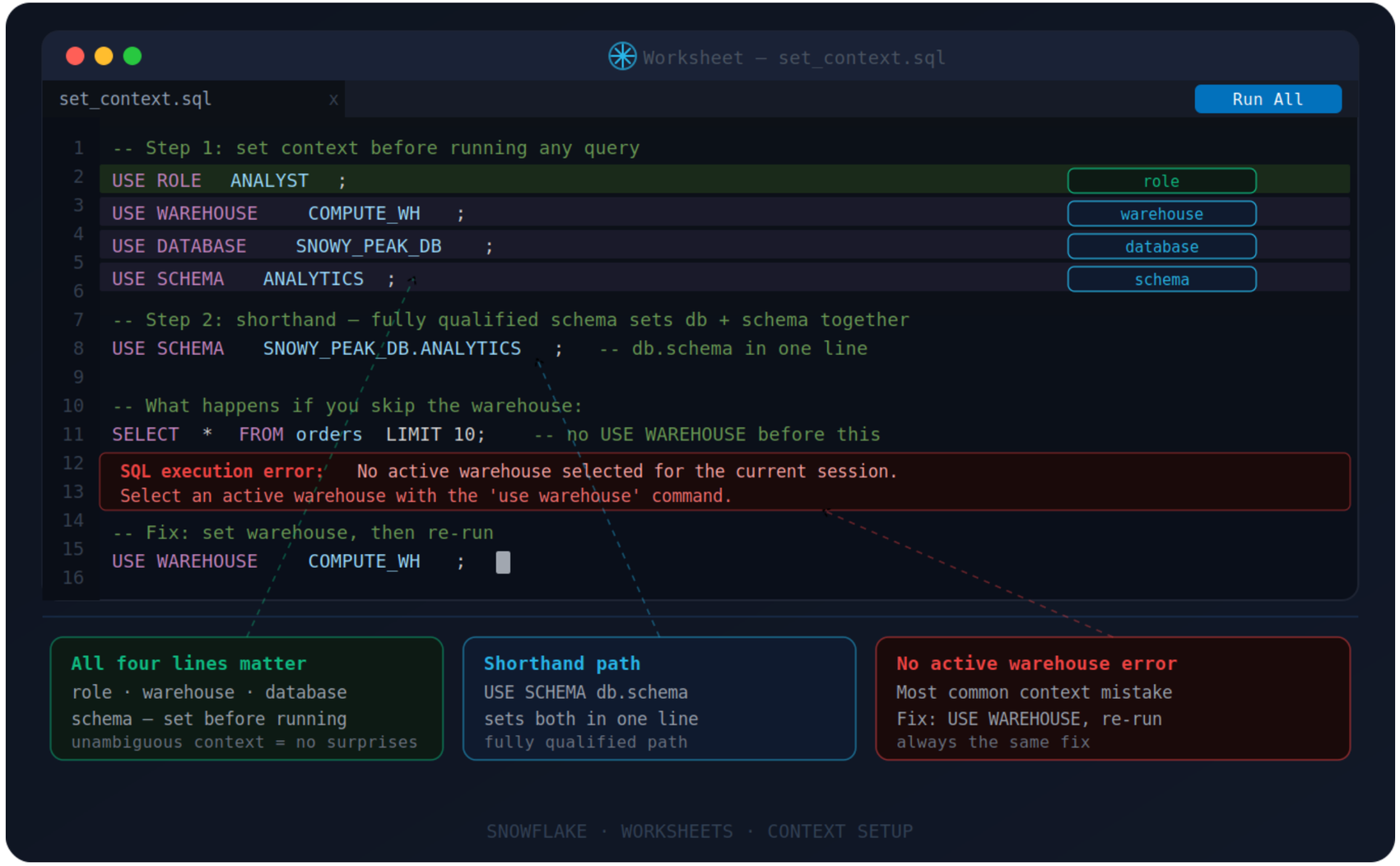


Creating a Schema Query

```
CREATE SCHEMA snowy_peak_db.raw;
```



Setting Context



Querying Your Data

Query

```
SELECT * EXCLUDE(email)
FROM user_registrations
LIMIT 10;
```

Output

Results (just now)

Table | Chart — 10 rows — 397ms

	USER_ID	FIRST_NAME	LAST_NAME	REGISTRATION_DATE	SUBSCRIPTION_TIER	COUNTRY	REFERRAL_SOURCE	APP_VERSION
	1001 – 1010	Camille 10.0%, Emma 10.0%, +8 more	Ashworth 10.0%, Bianchi 10.0%, +8 more	03/01/2024 – 07/02/2024	free 40.0%, premium 40.0%, +1 more	GB 30.0%, FR 20.0%, +3 more	organic 30.0%, app_store_search 20.0%, +3 more	3.2.2 50.0%, 3.2.1 30.0%, +1 more
1	1001	Emma	Thornton	2024-01-03	premium	GB	organic	3.2.1
2	1002	Luca	Ferretti	2024-01-07	free	IT	instagram	3.2.1
3	1003	Sophie	Marchand	2024-01-11	premium	FR	friend_referral	3.2.1
4	1004	James	Okafor	2024-01-15	free	GB	app_store_search	3.2.2
5	1005	Ingrid	Lindqvist	2024-01-19	pro	SE	instagram	3.2.2
6	1006	Marco	Bianchi	2024-01-22	free	IT	organic	3.2.2
7	1007	Hannah	Müller	2024-01-26	premium	DE	friend_referral	3.2.2
8	1008	Tom	Ashworth	2024-01-30	pro	GB	google_ad	3.2.2
9	1009	Camille	Dubois	2024-02-03	free	FR	organic	3.2.3
10	1010	Erik	Svensson	2024-02-07	premium	SE	app_store_search	3.2.3

- Limit does not guarantee a cheaper query

Understanding Your Tables

Query

```
SELECT table_name, row_count, bytes
FROM information_schema.tables
WHERE table_schema = 'PRODUCT';
```

Output

Results (just now)

Table | Chart

2 rows | 1.7s

	TABLE_NAME	ROW_COUNT	BYTES
1	IN_APP_PURCHASES	10	4096
2	APP_SESSIONS	10	4096

Understanding Your Columns

Query

```
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_name = 'IN_APP_PURCHASES';
```

Output

Results (just now)

Table | Chart | 10 rows | 1.7s

	COLUMN_NAME	DATA_TYPE	IS_NULLABLE
	AMOUNT_USD 10.0% CURRENCY 10.0% +8 more	TEXT 80.0% DATE 10.0% +1 more	YES 100.0%
1	PLATFORM	TEXT	YES
2	USER_ID	TEXT	YES
3	REGION	TEXT	YES
4	ITEM_TYPE	TEXT	YES
5	AMOUNT_USD	NUMBER	YES
6	PURCHASE_ID	TEXT	YES
7	STATUS	TEXT	YES
8	ITEM_NAME	TEXT	YES
9	PURCHASE_DATE	DATE	YES
10	CURRENCY	TEXT	YES

RBAC: Role-Based Access Control

Common Privileges

- `SELECT` to read data
- `INSERT`, `UPDATE` and `DELETE` to write data
- `USAGE` to access a database, schema, or warehouse
- `OWNERSHIP` for full structural control

RBAC: Granting Privileges in Practice

Granting access to the analyst role

```
GRANT USAGE ON DATABASE snowy_peak_db TO ROLE analyst;
GRANT USAGE ON SCHEMA product TO ROLE analyst;
GRANT SELECT ON TABLE product.in_app_purchases TO ROLE analyst;
```

- Users do not get privileges directly
  - They are assigned roles
- Principle of least privilege

Let's practice!

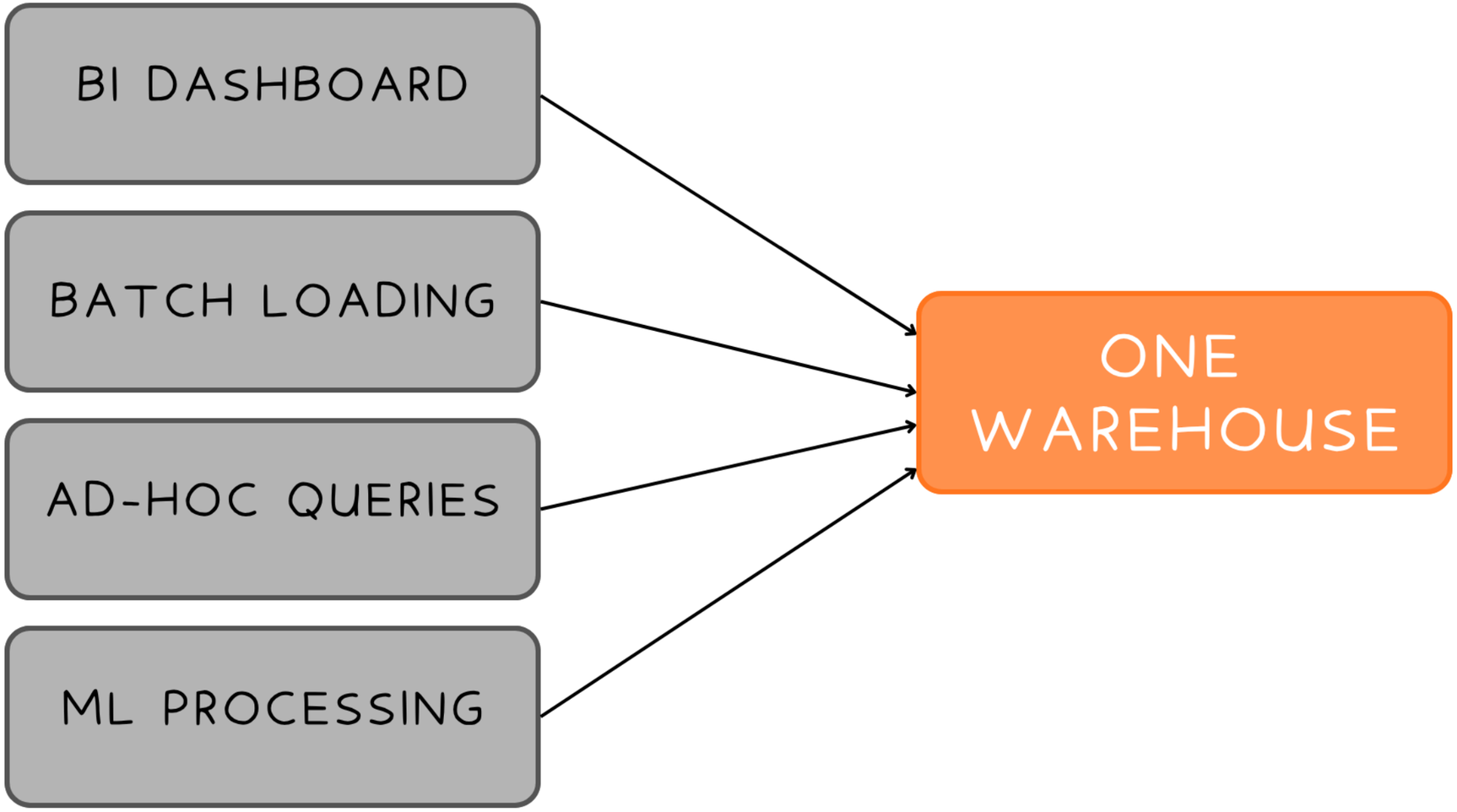
SNOWFLAKE ARCHITECTURE

Virtual Warehouses: Types, Sizing, and Scaling

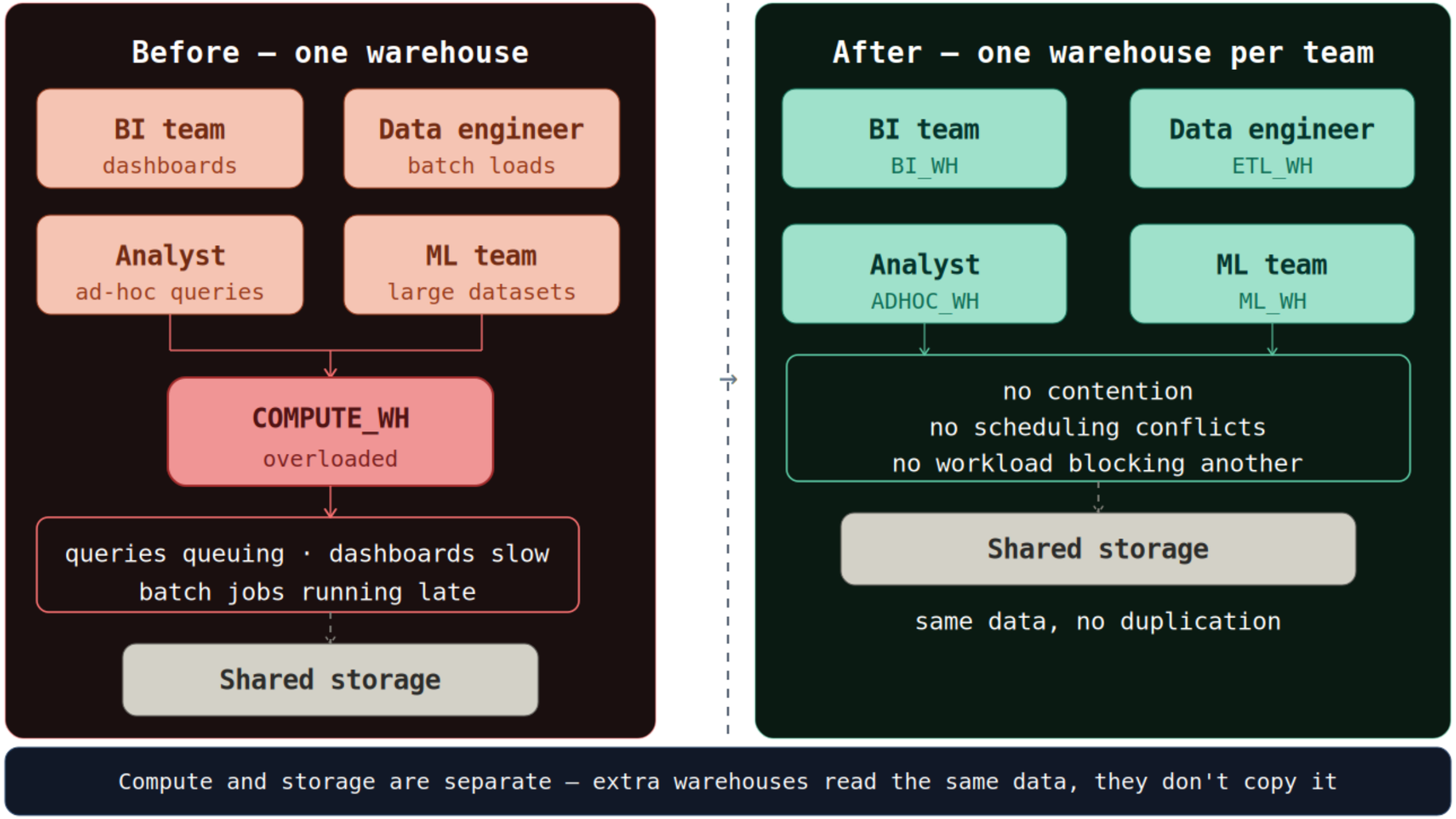
SNOWFLAKE ARCHITECTURE

Emily Melhuish
Technical Curriculum Developer, Snowflake

One Warehouse, Many Workloads



One Warehouse, Too Many Workloads



Standard vs Snowpark Optimized

Standard:

- Use cases:
  - SQL queries
  - Data loading
  - BI reporting
- Gen2 option available: Improved performance
- Test Gen1 vs Gen2 for your workload if cost matters

Snowpark Optimized

- Use cases:
  - ML model training
  - Snowpark DataFrames
  - Memory-intensive
- Available from Medium upward

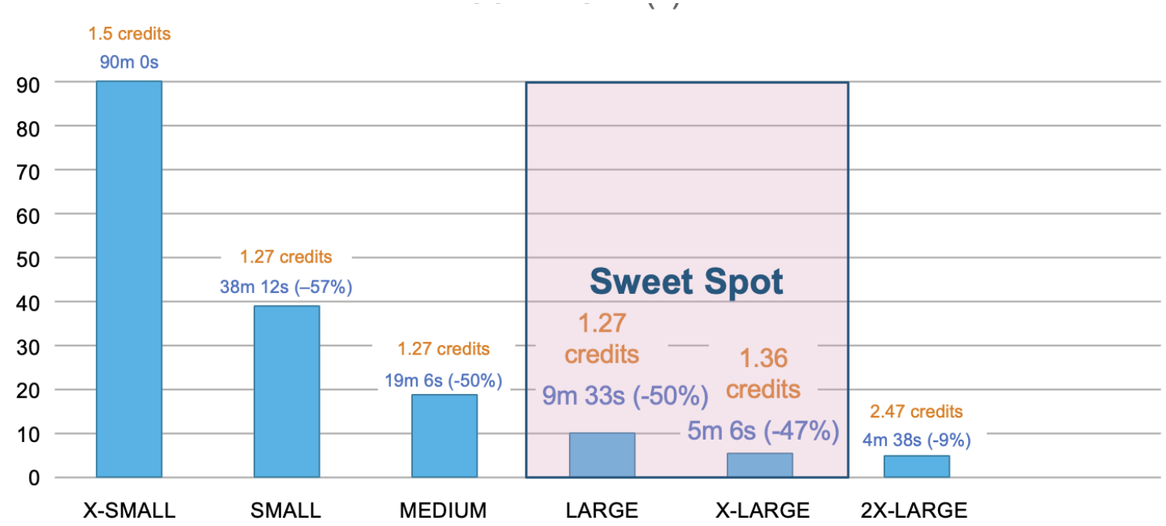
Warehouse Sizing

Warehouse size use cases:

- XS = Light queries
- XL = Large and Complex work

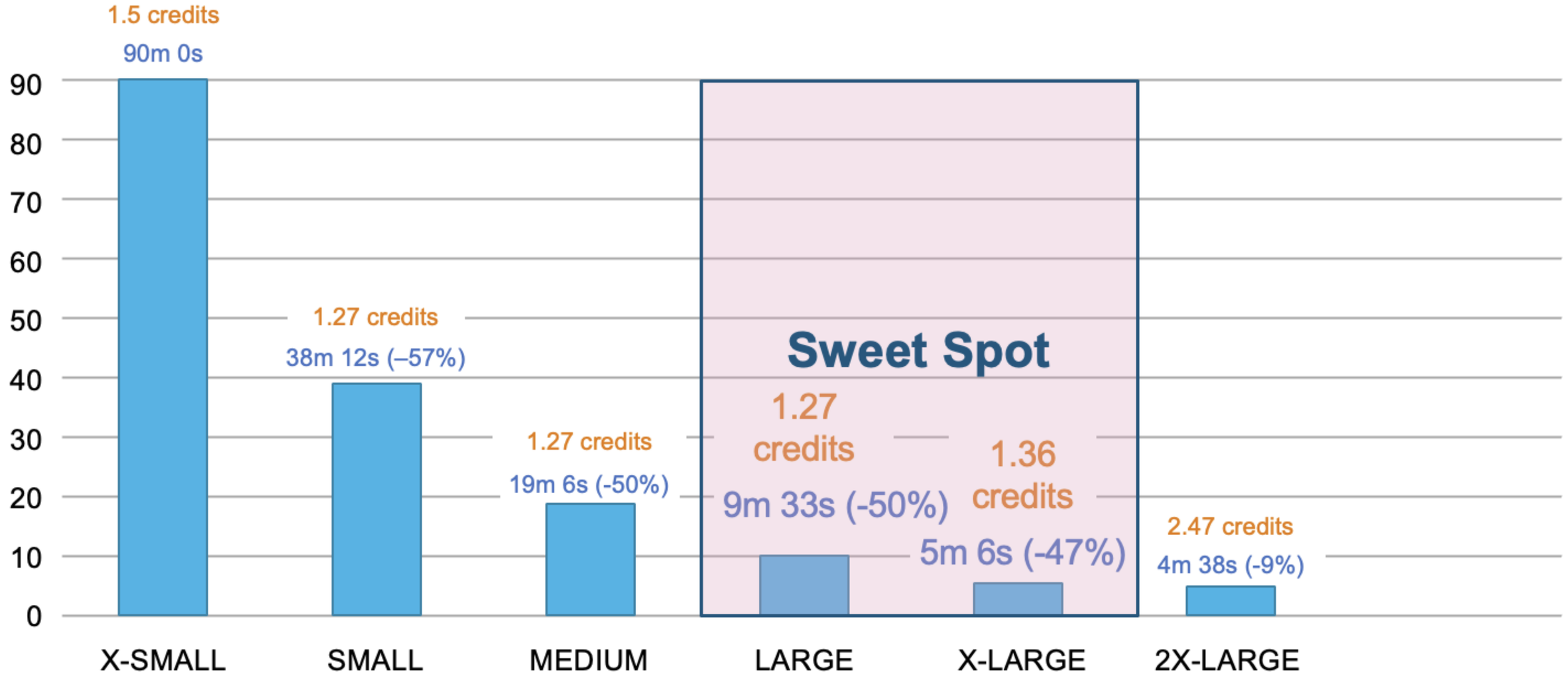
Query

```
CREATE WAREHOUSE analytics_wh
  WAREHOUSE_SIZE = 'LARGE'
  AUTO_SUSPEND = 300
  AUTO_RESUME = TRUE;
```



- bills per second with a 60-second minimum

The Sweet Spot



Cost Control: Auto-Suspend and Auto-Resume

Cost Control Setting Method 1

Auto-Suspend

- Defines seconds of inactivity before warehouse suspends
- Automatic suspension
- 300 seconds (5m) = DEFAULT

Cost Control Setting Method 2

Auto-Resume

- = TRUE means it wakes up when a query arrives
- No manual intervention required

Multi-cluster Warehouse

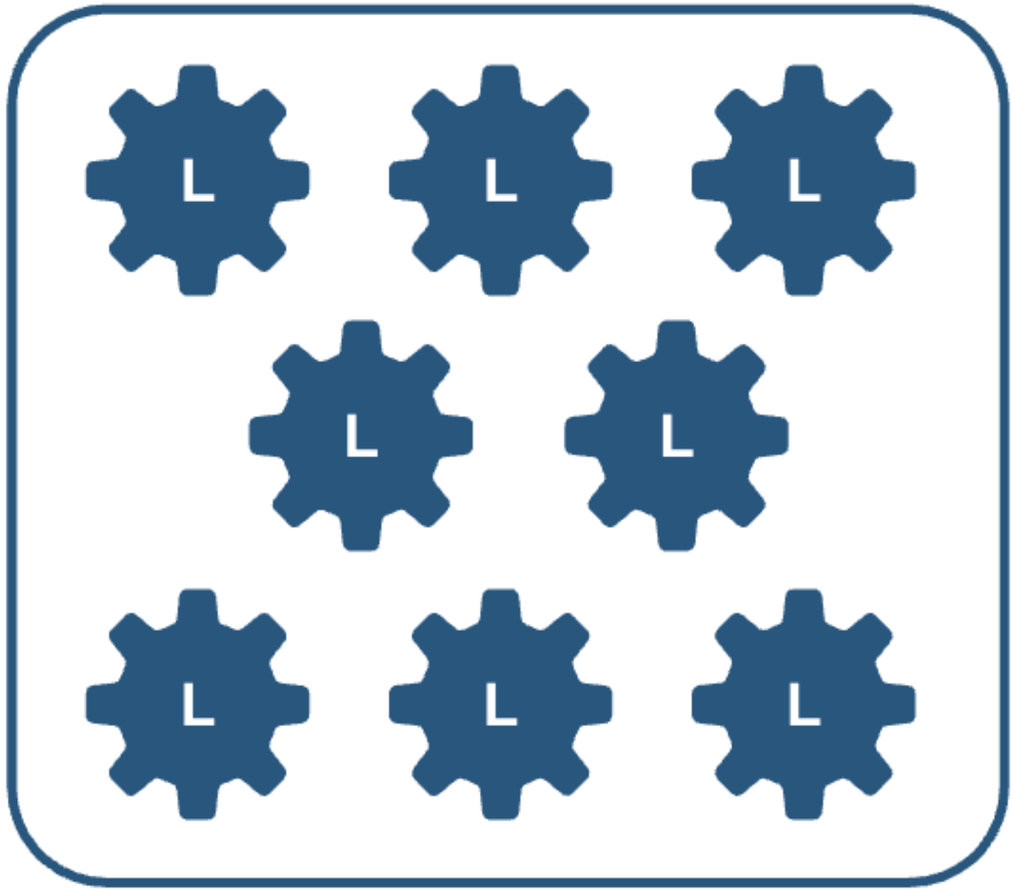
Use Case:

- Answer to high-concurrency workloads
- Clusters spin up/suspend automatically

Query

```
CREATE WAREHOUSE reporting_wh
  WAREHOUSE_SIZE = 'LARGE'
  MIN_CLUSTER_COUNT = 1
  MAX_CLUSTER_COUNT = 3
  SCALING_POLICY = 'STANDARD';
```

- Enterprise edition feature



Scaling Up vs Scaling Out

CREDITS PER HOUR

NUMBER OF CLUSTERS

VIRTUAL WAREHOUSE SIZE

	1	2	3	4	5	6	7	8	9	10
4XL	128	256	384	512	640	768	896	1024	1152	1280
3XL	64	128	192	256	320	384	448	512	576	640
2XL	32	64	96	128	160	192	224	256	288	320
XL	16	32	48	64	80	96	112	128	144	160
L	8	16	24	32	40	48	56	64	72	80
M	4	8	12	16	20	24	28	32	36	40
S	2	4	6	8	10	12	14	16	18	20
XS	1	2	3	4	5	6	7	8	9	10

XL x1 = 16 credits per hour
Scale up for complexity

M x4 = 16 credits per hour
Scale out for concurrency

Economy vs Standard Scaling Policy

Standard

- Spins up when anything is queued
- Throughput over savings
- Unpredictable Workloads

Economy

- Conservative
- Load will keep it busy >6 minutes
- Steady and predictable workloads

Let's practice!

SNOWFLAKE ARCHITECTURE

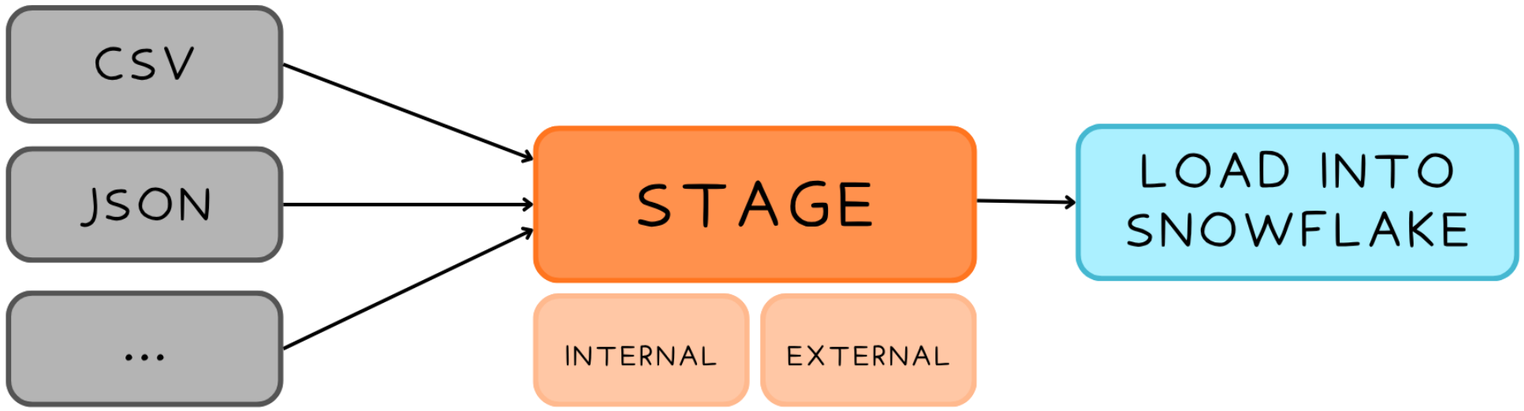
Loading Structured and Semi-Structured Data

SNOWFLAKE ARCHITECTURE

Emily Melhuish
Technical Curriculum Developer,
Snowflake

datacamp

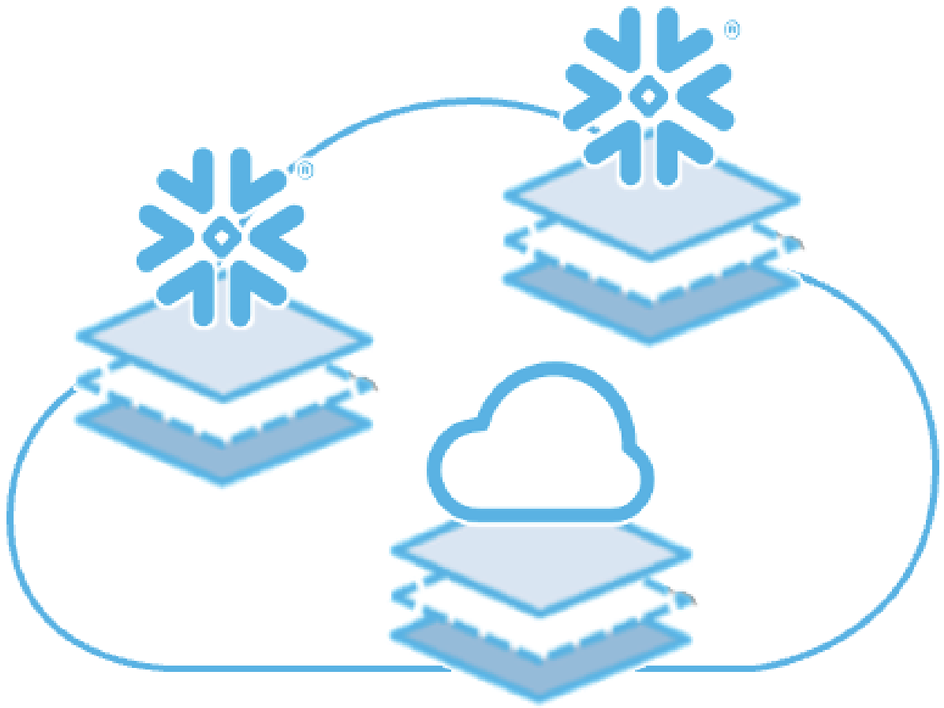
The Loading Workflow



- Files land in the stage, execute `COPY INTO` to load them

What is a Stage?

Stage = temporary storage location



Create Internal Stage

```
CREATE STAGE snowy_peak_stage;
```

Create External Stage

```
CREATE STAGE my_s3_stage
  STORAGE_INTEGRATION = s3_int
  URL = 's3://snowy-peak-data/raw/';
```

Loading data into Snowflake

COPY INTO

- Bulk loading from Stage
- Structured and Semi-Structured Data
- Scalable

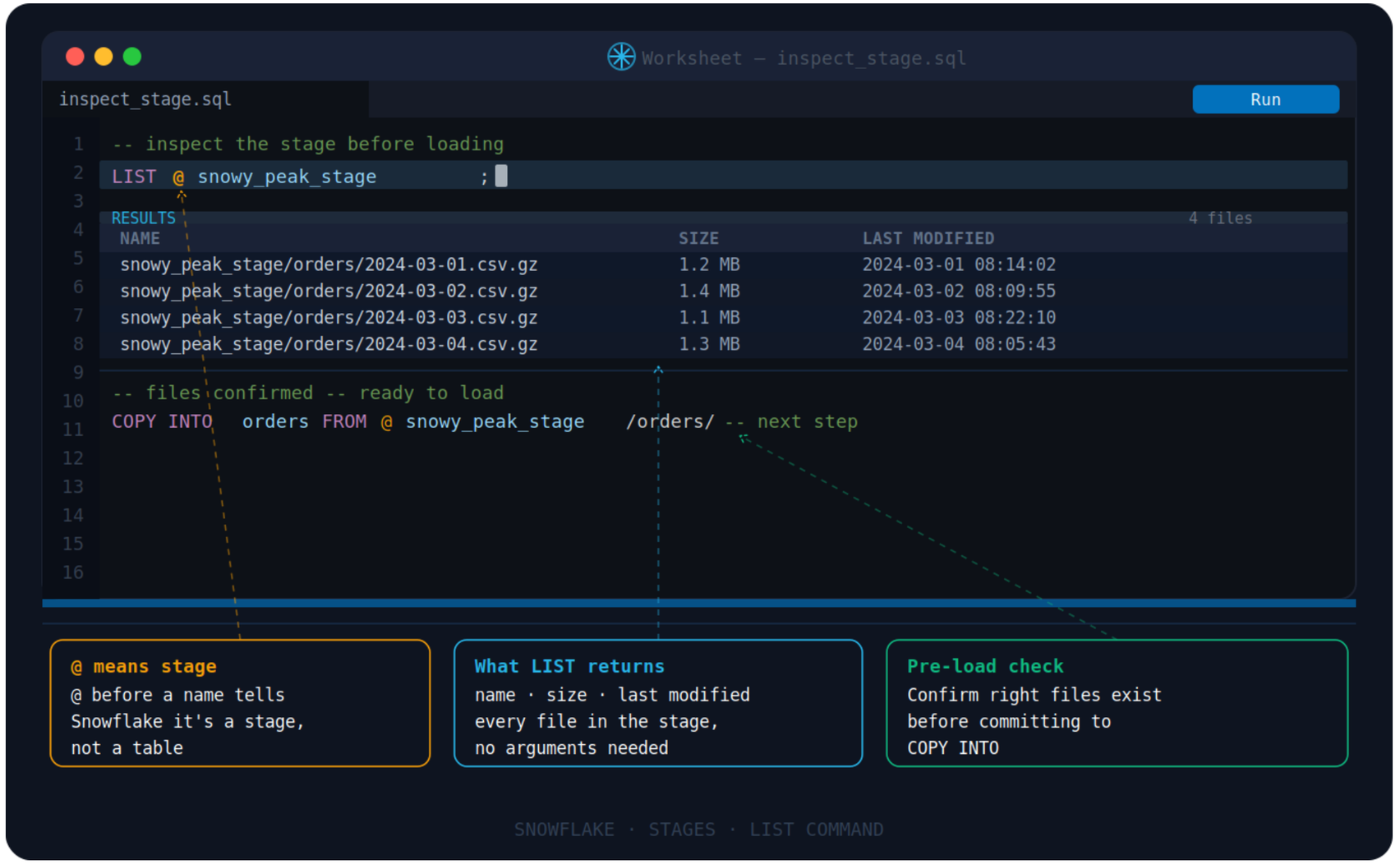
INSERT

- Great for a small number of rows without staging
- One-off additions or minor changes

Snowsight UI

- Non-technical users and quick one-off workloads

Inspecting a Stage with LIST



Loading Structured Data with COPY INTO

Query

```
CREATE OR REPLACE FILE FORMAT csv_format
  TYPE = CSV
  SKIP_HEADER = 1;

COPY INTO orders
FROM @snowy_peak_stage/orders.csv
FILE_FORMAT = (FORMAT_NAME = csv_format);
```

Loading and Querying Semi-Structured Data

Loading

JSON loads into a special Snowflake column type called VARIANT.

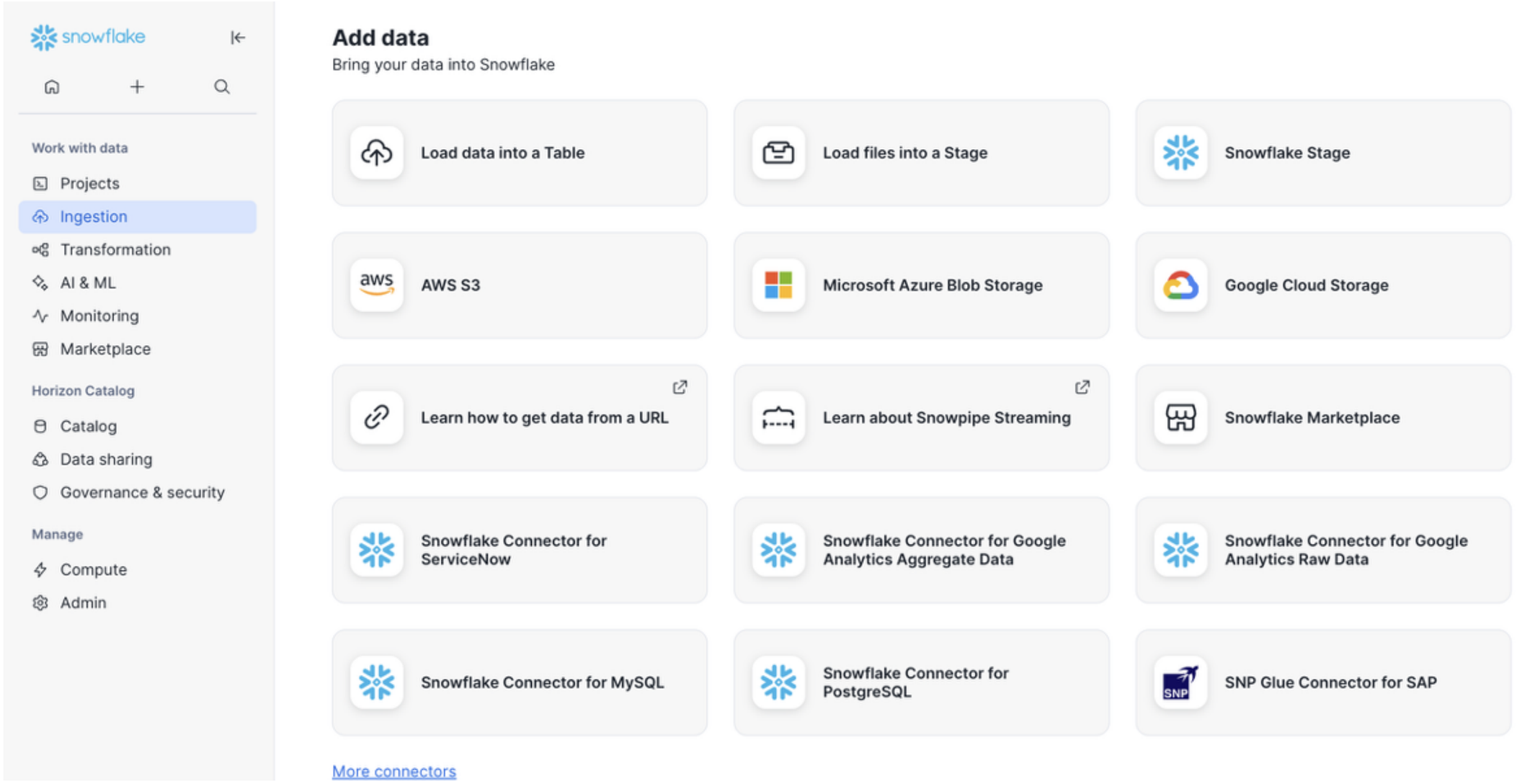
```
CREATE TABLE events (raw_event VARIANT);

COPY INTO events
FROM @snowy_peak_stage/events.json
FILE_FORMAT = (TYPE = JSON);
```

Querying

Use colon notation to navigate the JSON structure,

```
SELECT
raw_event:user_id::STRING AS user_id,
raw_event:event_type::STRING
AS event_type
FROM events;
```

Loading with the UI



Let's practice!

SNOWFLAKE ARCHITECTURE